

PolyGet: Manager Store, Discovery Catalog, and Repos Tab — Implementation Plan

For Antigravity: Implement task-by-task, in order. Each task should be independently testable before moving to the next. Do not skip the distro-detection task — later tasks depend on it.

Goal: Currently `discover_managers()` only surfaces package managers whose binary is already on `$PATH`, so the UI can never show "managers you don't have yet." This plan adds (1) a data-driven catalog of known managers (including ones without full driver classes), (2) auto-discovery of driver plugins instead of hardcoded imports, (3) distro-aware self-install commands so PolyGet can install a missing manager on the user's actual system, and (4) a Repos tab for managers that support repository/remote management (DNF repos+COPR, Flatpak remotes, Cargo registries, npm registries).

Architecture:

1. A new `distro.py` detects the running system's package-manager family from `/etc/os-release` (`fedora`, `arch`, `debian`, `suse`, `alpine`, `void`, `nix`, `unknown`).

2. A new `manager_catalog.yaml` + `catalog.py` loader lists every known manager as lightweight metadata (name, description, icon, binary, per-distro-family self-install command), independent of whether a full `PackageManager` driver class exists for it yet.
3. `drivers/__init__.py` is refactored from hardcoded imports to `pkgutil`-based auto-discovery, so dropping a new driver file in `app/core/drivers/` is enough to register it — no edits needed elsewhere.
4. `PackageManager` base class gains two new optional capability groups: self-install (`get_self_install_command(distro_family)`) and repo management (`supports_repos`, `list_repos()`, `get_add_repo_command()`, `get_remove_repo_command()`).
5. UI gains a "Manager Store" section (browse all catalog entries, Installed/Not Installed badge, Install button) and a "Repos" tab (only shows managers with `supports_repos = True`).

Tech Stack: Python 3, PySide6, PyYAML, asyncio
(unchanged — no new dependencies required)

Task 1: Distro Family Detection

Files:

- Create: `app/core/distro.py`
- Create: `tests/test_distro.py`

Step 1: Write the detection utility

```
"""Detect the running system's distro family
for choosing correct package-manager
commands."""
```

```
import functools
```

```
import re
```

```
_FAMILY_MAP = {
    "fedora": "fedora", "rhel": "fedora",
    "centos": "fedora",
    "arch": "arch", "cachyos": "arch",
    "manjaro": "arch", "endeavouros": "arch",
    "debian": "debian", "ubuntu": "debian",
    "suse": "suse", "opensuse": "suse",
    "alpine": "alpine",
    "void": "void",
    "nixos": "nix",
}
```

```
@functools.lru_cache(maxsize=1)
```

```
def get_distro_family() -> str:
```

```
    """Return a normalized distro family
    string by reading /etc/os-release.
```

```
    Checks ID first, then falls back to
    scanning ID_LIKE for a known family.
    Returns "unknown" if nothing matches.
    """
```

```
    try:
        with open("/etc/os-release", "r",
encoding="utf-8") as f:
            content = f.read()
    except OSError:
        return "unknown"
```

```
    fields = dict(re.findall(r'^(\w+)=(:|"
```

```
([^[^"]*)"|(.*))$', content, re.MULTILINE))
    id_val = (fields.get("ID", ("", ""))[0] or
fields.get("ID", ("", ""))[1] or "").lower()
    id_like = (fields.get("ID_LIKE", ("", ""))
[0] or fields.get("ID_LIKE", ("", ""))[1] or
 "").lower()

    if id_val in _FAMILY_MAP:
        return _FAMILY_MAP[id_val]
    for token in id_like.split():
        if token in _FAMILY_MAP:
            return _FAMILY_MAP[token]
    return "unknown"
```

Step 2: Write tests

Mock `/etc/os-release` content for Fedora, CachyOS (`ID=cachyos`, `ID_LIKE=arch`), Debian, and an unknown distro; assert correct family returned. Use `unittest.mock.mock_open` + `monkeypatch` on `open`, and clear the `lru_cache` between test cases via `get_distro_family.cache_clear()`.

Step 3: Verify

Run on your actual CachyOS machine and confirm it returns `"arch"`, not `"unknown"`.

Task 2: Manager Catalog (data-driven, includes managers with no driver class yet)

Files:

- Create: `app/core/data/manager_catalog.yaml`

- Create: `app/core/catalog.py`
- Create: `tests/test_catalog.py`

Step 1: Define the catalog schema

```
# app/core/data/manager_catalog.yaml
- id: dnf
  name: DNF
  category: System
  description: Fedora/RHEL system package
  manager:
    icon: package-x-generic
    binary: dnf
    has_driver: true
    self_install: {} # already the system's own
  manager on fedora family; nothing to install

- id: pacman
  name: Pacman
  category: System
  description: Arch Linux system package
  manager:
    icon: package-x-generic
    binary: pacman
    has_driver: false
    self_install: {}

- id: paru
  name: Paru (AUR helper)
  category: System
  description: AUR helper for Arch-based
  systems, wraps pacman for community packages.
  icon: package-x-generic
  binary: paru
  has_driver: false
  self_install:
```

```

    arch: ["sudo", "pacman", "-S", "--noconfirm", "paru"]

- id: flatpak
  name: Flatpak
  category: Universal
  description: Sandboxed, distro-agnostic universal app packages.
  icon: emblem-downloads
  binary: flatpak
  has_driver: true
  self_install:
    fedora: ["pkexec", "dnf", "install", "-y", "flatpak"]
    arch: ["sudo", "pacman", "-S", "--noconfirm", "flatpak"]
    debian: ["pkexec", "apt-get", "install", "-y", "flatpak"]
    suse: ["pkexec", "zypper", "install", "-y", "flatpak"]

- id: npm
  name: NPM
  category: Language/Dev
  description: Node.js package manager for globally installed JS tooling.
  icon: text-x-script
  binary: npm
  has_driver: true
  self_install:
    fedora: ["pkexec", "dnf", "install", "-y", "nodejs"]
    arch: ["sudo", "pacman", "-S", "--noconfirm", "nodejs", "npm"]
    debian: ["pkexec", "apt-get", "install", "-y", "nodejs", "npm"]

```

```
- id: pipx
  name: Pipx
  category: Language/Dev
  description: Installs and runs Python CLI
applications in isolated environments.
  icon: text-x-python
  binary: pipx
  has_driver: true
  self_install:
    fedora: ["pkexec", "dnf", "install", "-y",
"pipx"]
    arch: ["sudo", "pacman", "-S", "--
noconfirm", "python-pipx"]
    debian: ["pkexec", "apt-get", "install",
"-y", "pipx"]

- id: cargo
  name: Cargo
  category: Language/Dev
  description: Rust toolchain's package
manager, for globally installed Rust binaries.
  icon: text-x-generic
  binary: cargo
  has_driver: true
  self_install:
    fedora: ["pkexec", "dnf", "install", "-y",
"cargo"]
    arch: ["sudo", "pacman", "-S", "--
noconfirm", "rust"]
    debian: ["pkexec", "apt-get", "install",
"-y", "cargo"]

- id: apt
  name: APT
  category: System
```

```
description: Debian/Ubuntu system package
manager.
icon: package-x-generic
binary: apt-get
has_driver: false
self_install: {}

- id: zypper
  name: Zypper
  category: System
  description: openSUSE system package
manager.
icon: package-x-generic
binary: zypper
has_driver: false
self_install: {}

- id: apk
  name: APK
  category: System
  description: Alpine Linux system package
manager.
icon: package-x-generic
binary: apk
has_driver: false
self_install: {}

- id: xbps
  name: XBPS
  category: System
  description: Void Linux system package
manager.
icon: package-x-generic
binary: xbps-install
has_driver: false
self_install: {}
```

```
- id: nix
  name: Nix
  category: Universal
  description: Purely functional, reproducible
package manager and profile system.
  icon: package-x-generic
  binary: nix
  has_driver: false
  self_install: {}

- id: snap
  name: Snap
  category: Universal
  description: Canonical's sandboxed universal
app packages.
  icon: emblem-downloads
  binary: snap
  has_driver: false
  self_install:
    fedora: ["pkexec", "dnf", "install", "-y",
"snapt"]
    debian: ["pkexec", "apt-get", "install",
"-y", "snapt"]

- id: distrobox
  name: Distrobox
  category: Universal
  description: Runs containerized versions of
other distros' package managers on your host.
  icon: utilities-terminal
  binary: distrobox
  has_driver: false
  self_install:
    fedora: ["pkexec", "dnf", "install", "-y",
"distrobox"]
```

```
arch: ["sudo", "pacman", "-S", "--noconfirm", "distrobox"]
```

Step 2: Write the loader

```
"""Loads the static manager catalog for the
Manager Store, independent of installed
drivers."""

from dataclasses import dataclass, field
from importlib import resources
import yaml

from app.core.manager import discover_managers
from app.core.distro import get_distro_family

@dataclass
class CatalogEntry:
    id: str
    name: str
    category: str
    description: str
    icon: str
    binary: str
    has_driver: bool
    self_install: dict[str, list[str]] =
field(default_factory=dict)
    installed: bool = False # populated at
runtime

    def get_self_install_command(self) ->
list[str] | None:
    """Return the install command for the
current distro family, if known."""
```

```

        family = get_distro_family()
        return self.self_install.get(family)

def load_catalog() -> list[CatalogEntry]:
    """Load the static manager catalog and
    annotate each entry with installed status."""
    with
    resources.files("app.core.data").joinpath("man
    ager_catalog.yaml").open("r", encoding="utf-
    8") as f:
        raw = yaml.safe_load(f) or []

        active_names = {mgr.name.lower() for mgr
        in discover_managers()}
        entries = []
        for item in raw:
            entry = CatalogEntry(**item)
            entry.installed = entry.name.lower()
            in active_names or
            _binary_present(entry.binary)
            entries.append(entry)
        return entries

def _binary_present(binary: str) -> bool:
    import shutil
    return shutil.which(binary) is not None

```

Step 3: Write tests

`tests/test_catalog.py` : assert `load_catalog()`
returns ≥ 14 entries, every entry has a non-empty
`id / name`, and `get_self_install_command()` returns
`None` gracefully for a distro family with no mapping (e.g.
`alpine` on the `paru` entry).

Step 4: Package the data directory

Ensure `app/core/data/` has an `__init__.py` (even if empty) so

```
importlib.resources.files("app.core.data")
```

resolves correctly, and add it to any packaging config (`MANIFEST.in` / `pyproject.toml` `package-data`) if one exists.

Task 3: Auto-Discovery of Driver Plugins

Files:

- Modify: `app/core/drivers/__init__.py`
- Modify: `tests/test_drivers_installed.py` (add a discovery-count assertion)

Step 1: Replace hardcoded imports with pkgutil scanning

```
"""Package manager driver implementations.

Importing this module auto-discovers and
registers every driver module in this package,
so adding a new driver file here is sufficient
– no manual import needed.
"""

import importlib
import pkgutil

for _, module_name, _ in
    pkgutil.iter_modules(__path__):
```

```
importlib.import_module(f"__name__.{
module_name}")
```

Step 2: Verify existing five drivers still register

Run `python -c "from app.core.manager import discover_managers; print([m.name for m in discover_managers()])"` and confirm the same managers appear as before the refactor.

Step 3: Regression test

Add a test asserting `len(_REGISTRY) >= 5` after import, so a future accidental deletion of a driver file (or a broken new one) is caught.

Task 4: Self-Install Support on the Base Class

Files:

- Modify: `app/core/manager.py`

Step 1: Add an optional method to `PackageManager`

```
def get_self_install_command(self) ->
list[str] | None:
    """Get the command to install this
    manager itself on the current distro.

    Returns:
        list[str] | None: Command list, or
        None if unknown/unsupported for this distro.
    """
    return None
```

This is intentionally on the base class as a no-op default
— individual drivers don't need to implement it since the catalog (Task 2) is the primary source of self-install commands for managers without a driver. Drivers *with* a class (DNF, Flatpak, etc.) can override it directly if you'd rather keep the command next to the driver than in YAML
— pick one source of truth and note the choice in a code comment so it doesn't drift.

Task 5: Repo Support on the Base Class + First Two Implementations

Files:

- Modify: `app/core/manager.py`
- Modify: `app/core/drivers/dnf.py`
- Modify: `app/core/drivers/flatpak.py`
- Create: `tests/test_repos.py`

Step 1: Extend the base class

```
supports_repos: bool = False

async def list_repos(self) ->
list[dict[str, Any]]:
    """List configured
    repositories/remotes for this manager.

    Returns:
        list[dict[str, Any]]: Each dict
        has at least 'id', 'name', 'url', 'enabled'.
        """
```

```

        raise NotImplementedError("This
manager does not support repo listing")

    def get_add_repo_command(self,
repo_url_or_id: str) -> list[str]:
        """Get the command to add a
repository/remote."""
        raise NotImplementedError("This
manager does not support adding repos")

    def get_remove_repo_command(self, repo_id:
str) -> list[str]:
        """Get the command to remove a
repository/remote."""
        raise NotImplementedError("This
manager does not support removing repos")

```

Step 2: Implement for DNF (repos + COPR)

```

        supports_repos: bool = True

    async def list_repos(self) ->
list[dict[str, Any]]:
        proc = await
asyncio.create_subprocess_exec(
            "dnf", "repolist", "--all", "--
quiet",
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE
        )
        stdout, _ = await proc.communicate()
        repos = []
        for line in
stdout.decode(errors="ignore").splitlines()
[1:]:

```

```

        parts = line.split(None, 2)
        if len(parts) >= 2:
            enabled = not
parts[0].startswith("!") and not
parts[0].startswith("*")
            repo_id =
parts[0].lstrip("!*")
            repos.append({"id": repo_id,
"name": parts[1] if len(parts) > 1 else
repo_id,
                                "url": "",
"enabled": enabled})
        return repos

    def get_add_repo_command(self,
repo_url_or_id: str) -> list[str]:
        if repo_url_or_id.count("/") == 1 and
not repo_url_or_id.startswith("http"):
            # looks like a COPR shorthand,
e.g. "username/project"
            return ["pkexec", "dnf", "copr",
"enable", "-y", repo_url_or_id]
            return ["pkexec", "dnf", "config-
manager", "--add-repo", repo_url_or_id]

    def get_remove_repo_command(self, repo_id:
str) -> list[str]:
        return ["pkexec", "dnf", "config-
manager", "--set-disabled", repo_id]

```

Step 3: Implement for Flatpak (remotes)

```

supports_repos: bool = True

async def list_repos(self) ->

```

```

list[dict[str, Any]]:
    proc = await
asyncio.create_subprocess_exec(
    "flatpak", "remotes", "--
columns=name,url,disabled",
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE
)
    stdout, _ = await proc.communicate()
    repos = []
    for line in
stdout.decode(errors="ignore").splitlines():
        parts = line.split("\t")
        if len(parts) >= 2:
            repos.append({"id": parts[0],
"name": parts[0], "url": parts[1],
                        "enabled":
(parts[2].strip().lower() != "yes") if
len(parts) > 2 else True})
    return repos

    def get_add_repo_command(self,
repo_url_or_id: str) -> list[str]:
        if repo_url_or_id.strip().lower() in
("flathub",
"https://dl.flathub.org/repo/flathub.flatpakre
po"):
            return ["flatpak", "remote-add",
"--if-not-exists", "flathub",

"https://dl.flathub.org/repo/flathub.flatpakre
po"]
        return ["flatpak", "remote-add", "--
if-not-exists", repo_url_or_id,
repo_url_or_id]

```

```
def get_remove_repo_command(self, repo_id:
str) -> list[str]:
    return ["flatpak", "remote-delete",
repo_id]
```

Leave Pipx, NPM, and Cargo with `supports_repos = False` for now (npm/cargo registry config is a reasonable Phase 2 follow-up but has different semantics – per-project vs global – so don't rush it into this pass).

Step 4: Tests

Mock subprocess output for `dnf repolist` and `flatpak remotes`, assert `list_repos()` parses correctly; assert `get_add_repo_command()` picks the COPR path vs the generic path correctly.

Task 6: UI – Manager Store Section

Files:

- Modify: `app/ui/main_window.py`

Step 1: Add a "Manager Store" entry to the nav sidebar

Next to the existing "Browse Store" (package search) item, add a second nav item, e.g. `item_manager_store = QListWidgetItem("Manager Store")` with a distinct icon (`QIcon.fromTheme("preferences-system")`).

Step 2: Build the page

List every `CatalogEntry` from `load_catalog()`. Reuse the existing `StoreItemWidget` pattern (or a lightweight variant) showing: name, description, category, and an

Installed badge vs an "Install" button. If `has_driver` is `False`, add a small note like "Browsing only — package management support coming later" so expectations are clear.

Step 3: Wire the Install button

On click, resolve `entry.get_self_install_command()`. If `None` (no mapping for this distro), disable the button and show "Not available for your distro yet" instead of silently failing. Otherwise run it through your existing `pkexec/sudo` worker pattern (same one used for DNF upgrades), and on success, refresh the catalog (`installed` flags) and the main manager list.

Task 7: UI — Repos Tab

Files:

- Modify: `app/ui/main_window.py`

Step 1: Add a "Repos" nav item

Only meaningful once at least one installed manager has `supports_repos = True`.

Step 2: Build the page

Sidebar lists installed managers where

`mgr.supports_repos` is `True` (initially just DNF and Flatpak, once installed). Main panel shows that manager's repos via `await mgr.list_repos()`, each with an enable/disable or remove action, plus an "Add Repo" input box wired to `mgr.get_add_repo_command()`.

Step 3: Reuse existing worker/coordinator infrastructure

Repo add/remove commands should go through the same `SubprocessCoordinator`-tracked worker pattern as upgrades, so they can be cancelled and don't leak processes.

Task 8: Wrap-up Tests

Files:

- Modify: `tests/test_catalog.py`
- Modify: `tests/test_repos.py`

Confirm end-to-end: `load_catalog()` on a mocked "arch" distro family surfaces a working `paru` self-install command, and `list_repos() / get_add_repo_command()` round-trip correctly for both DNF and Flatpak using subprocess mocks (no real system calls in tests).